EELU
الجامعة المصرية للتعلم الإلكتروني الأهلية
THE EGYPTIAN E-LEARNING UNIVERSITY

2026-2027

Information Technology

Computer Graphics

Section (3)

Today's Objectives

1

Orthogonal Projection

Matrix stacks, and how **gluOrtho2D** sets the viewing rectangle.

2

2D Transformations

Move, rotate and scale shapes with **glTranslatef**, **glRotatef** and **glScalef**.

3

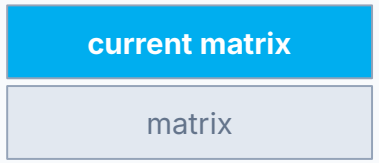
Keyboard Control

Move and rotate the shape live, using the keyboard.

Orthogonal Projection in OpenGL

OpenGL Is Stack-Based

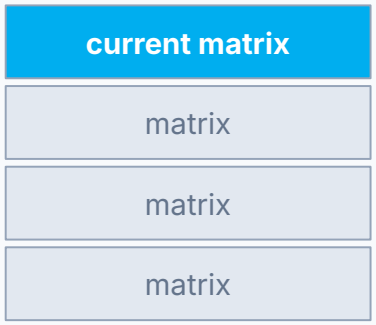
GL_PROJECTION



Depth: at least 2

How the scene is projected onto the window.

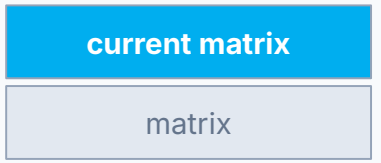
GL_MODELVIEW



Depth: at least 32

Transformations applied to our shapes.

GL_TEXTURE



Depth: at least 2

Texture coordinates (not used in this lab).

glMatrixMode(mode) selects which stack we are changing. Every matrix call then affects the **top matrix of the selected stack**.

Setting Up the Projection

```
void SetTransformations() {
    // 1. work on the projection stack
    glMatrixMode(GL_PROJECTION);
    // 2. clear it (identity matrix)
    glLoadIdentity();
    // 3. Left, Right, Bottom, Top
    gluOrtho2D(-100, 100, -100, 100);
    // 4. back to the modelview stack
    glMatrixMode(GL_MODELVIEW);
}
```

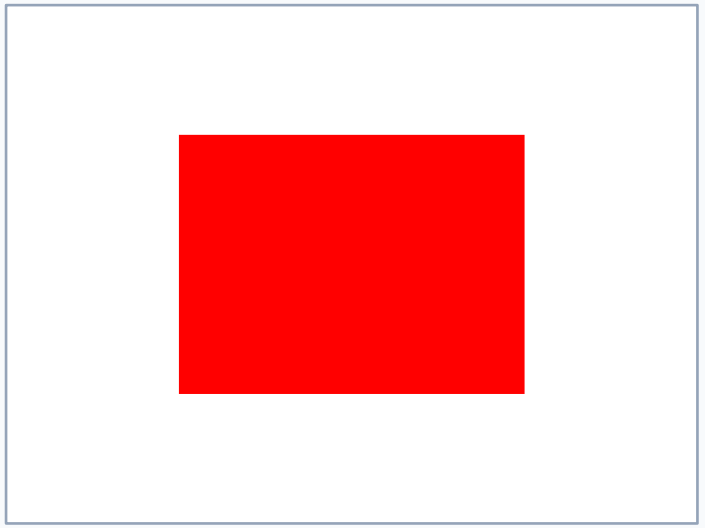
gluOrtho2D sets the **maximum viewing rectangle**: the part of the world shown in the window.

1. **glMatrixMode**
   Selects the projection stack.
2. **glLoadIdentity**
   Clears it, so old values do not pile up.
3. **gluOrtho2D**
   Sets the viewing rectangle.
4. **glMatrixMode**
   Returns to MODELVIEW for drawing.

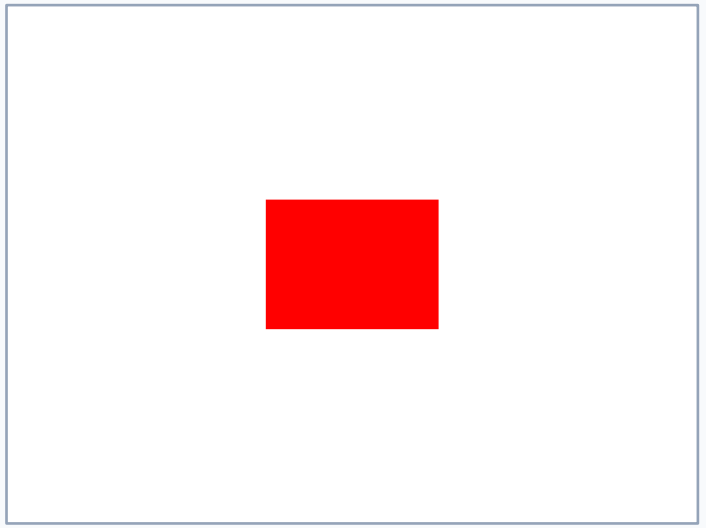
Follow along

Experiment: Viewing Rectangle

gluOrtho2D(-100, 100, -100, 100)

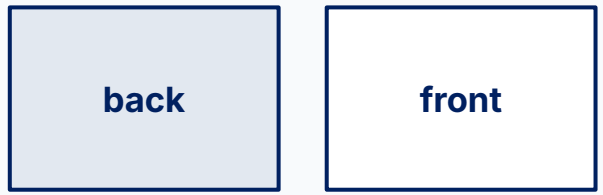


gluOrtho2D(-200, 200, -200, 200)



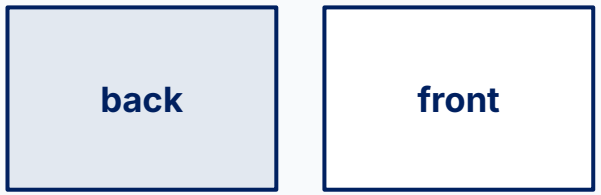
Try it: the red square is 100 units wide. A bigger viewing rectangle makes it look **smaller**. It looks wide because the window is 800×600 but the range is square.

Double Buffering



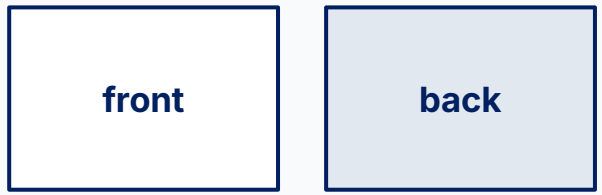
1. Draw

OpenGL draws the new frame into the hidden **back buffer**.



2. Swap

glutSwapBuffers() swaps back and front buffers.



3. Show

The finished frame appears on screen: no flicker.

Why we need it: key presses change the shape many times per second. We draw into the back buffer (**GLUT_DOUBLE**) and call **glutSwapBuffers()** at the end of **OnDisplay**.

Follow along

Step 1 (1/3): Headers & main()

Headers & prototypes

```
#include <windows.h>
#include <stdlib.h>
#include <gl/gl.h>
#include <gl/glut.h>

void InitGraphics(int argc, char *argv[]);
void SetTransformations();
void OnDisplay();
```

main()

```
int main(int argc, char* argv[]) {
    InitGraphics(argc, argv);
    return 0;
}
```

Follow along

Step 1 (2/3): InitGraphics & Projection

InitGraphics()

```
void InitGraphics(int argc, char *argv[]) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA);
    glutInitWindowPosition(100, 100);
    glutInitWindowSize(800, 600);
    glutCreateWindow("OpenGL Lab 3");
    glutDisplayFunc(OnDisplay);
    SetTransformations();
    glutMainLoop();
}
```

SetTransformations()

```
void SetTransformations() {
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-100, 100, -100, 100);
    glMatrixMode(GL_MODELVIEW);
}
```

Follow along

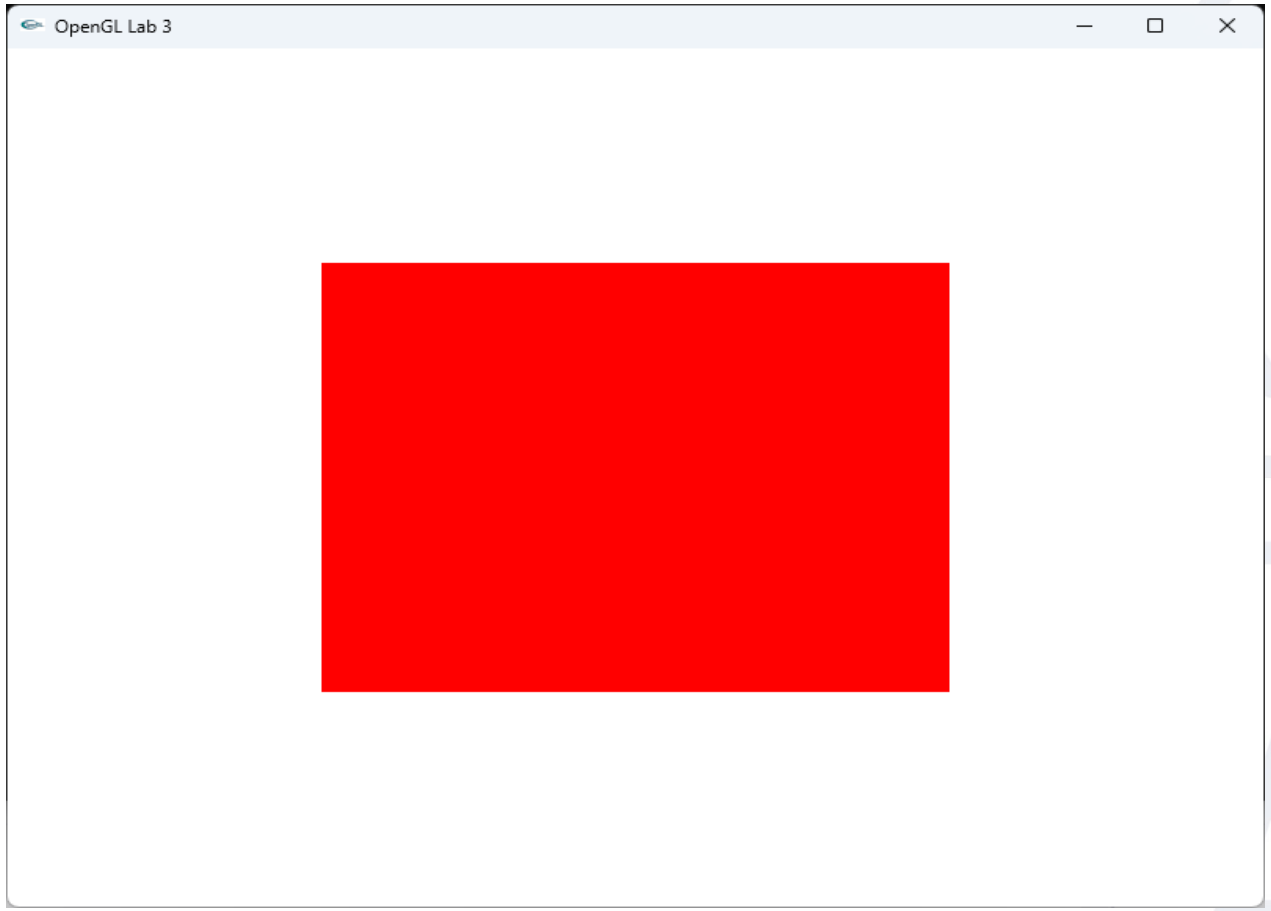
Step 1 (3/3): OnDisplay

```
void OnDisplay() {
    glLoadIdentity();
    glClearColor(1, 1, 1, 1);
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_QUADS);
        glColor3f(1, 0, 0);
        glVertex3f(-50, -50, 0);
        glVertex3f(50, -50, 0);
        glVertex3f(50, 50, 0);
        glVertex3f(-50, 50, 0);
    glEnd();
    glutSwapBuffers();
}
```

What it does

- **glLoadIdentity()** resets the modelview matrix.
- Clears the window to **white**.
- Draws a **red** 100×100 square centered on (0, 0).
- **glutSwapBuffers()** shows the finished frame.

Run: Static Red Square

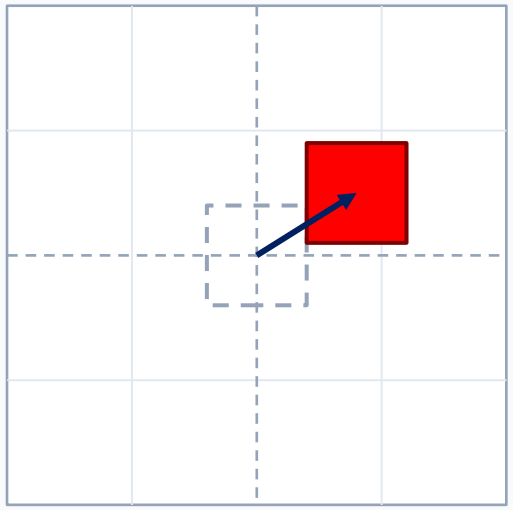


2D Transformations

Translate · Rotate · Scale

2D Transformations at a Glance

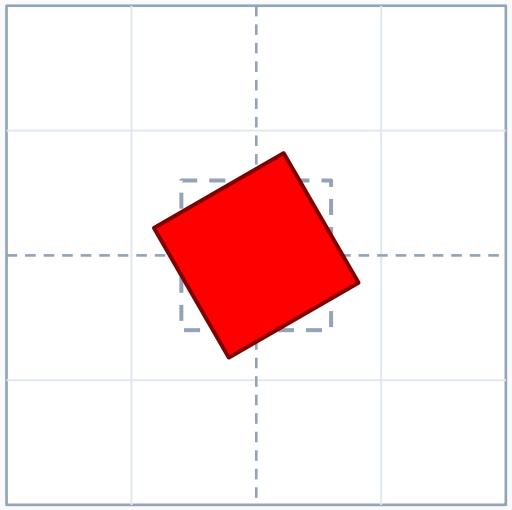
glTranslatef



```
glTranslatef(40, 25, 0);
```

Moves by x and y (z = 0).

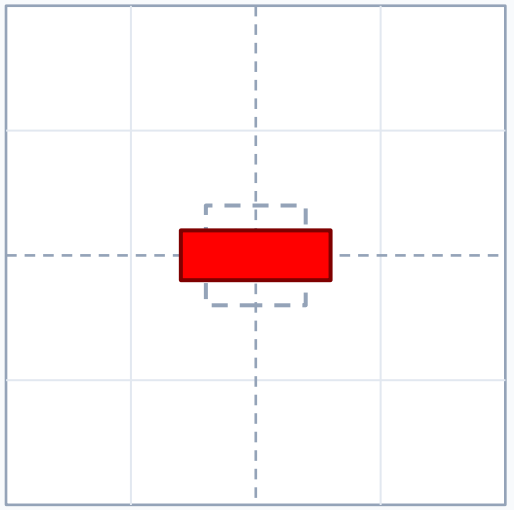
glRotatef



```
glRotatef(30, 0, 0, 1);
```

Counter-clockwise, around the origin.

glScalef



```
glScalef(1.5, 0.5, 1);
```

Stretches x and y (z = 1).

Keyboard Control

Follow along

Step 2: Add Translation

```
// global variables (top of the file)
float fXPos = 0, fYPos = 0, fRot = 0;
void OnDisplay() {
    glLoadIdentity();
    glClearColor(1, 1, 1, 1);
    glClear(GL_COLOR_BUFFER_BIT);
    glTranslatef(fXPos, fYPos, 0);   // NEW
    // ... draw the square ...
```

What changed

- Globals hold the position and angle.
- The keys will change them next.

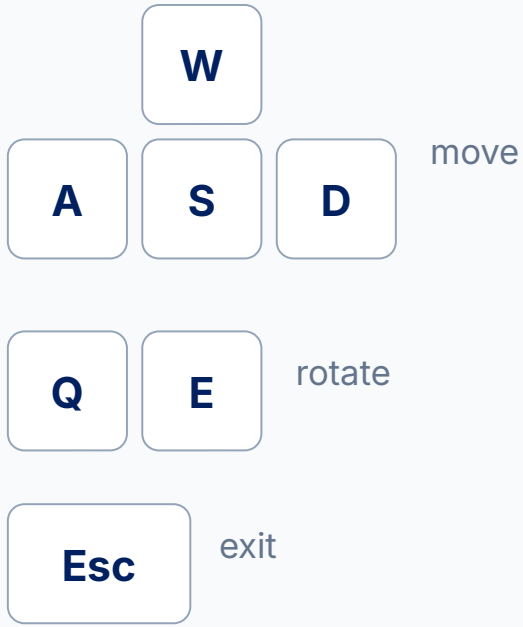
Declare the globals **above** all functions.

Follow along

Code: OnKeyPress

```
void OnKeyPress(unsigned char key, int x, int y) {
    if (key == 27)
        exit(0);
    switch (key) {
        case 'a': case 'A': fXPos -= 0.5; break;
        case 'd': case 'D': fXPos += 0.5; break;
        case 'w': case 'W': fYPos += 0.5; break;
        case 's': case 'S': fYPos -= 0.5; break;
        case 'e': case 'E': fRot += 0.1;  break;
        case 'q': case 'Q': fRot -= 0.1;  break;
    }
}
```

Keys



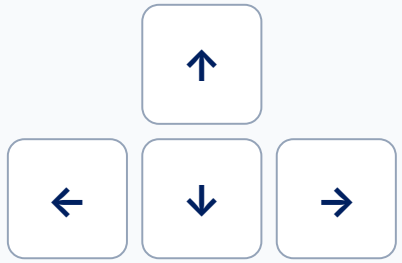
Follow along

Code: OnSpecialKeyPress

```
void OnSpecialKeyPress(int key, int x, int y) {
    switch (key) {
        case GLUT_KEY_LEFT:  fXPos -= 0.5; break;
        case GLUT_KEY_RIGHT: fXPos += 0.5; break;
        case GLUT_KEY_UP:    fYPos += 0.5; break;
        case GLUT_KEY_DOWN:  fYPos -= 0.5; break;
    }
}
```

The arrows do the same job as **W A S D**.

Arrow keys



Move the square by 0.5 units per press.
Hold the key to repeat.

Follow along

Register the Callbacks

```
void InitGraphics(int argc, char *argv[]) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA);
    glutInitWindowPosition(100, 100);
    glutInitWindowSize(800, 600);
    glutCreateWindow("OpenGL Lab 3");
    glutDisplayFunc(OnDisplay);
    glutIdleFunc(OnDisplay);          // NEW
    glutKeyboardFunc(OnKeyPress);     // NEW
    glutSpecialFunc(OnSpecialKeyPress); // NEW
    SetTransformations();
    glutMainLoop();
}
```

Add 2 prototypes: **OnKeyPress**, **OnSpecialKeyPress**.

Why glutIdleFunc?

- It keeps redrawing the window, so every key press shows up **immediately**.

Follow along

Step 3: Add Rotation

```
void OnDisplay() {
    glLoadIdentity();
    glClearColor(1, 1, 1, 1);
    glClear(GL_COLOR_BUFFER_BIT);
    glTranslatef(fXPos, fYPos, 0);
    glRotatef(fRot, 0, 0, 1);       // NEW
    glBegin(GL_QUADS);
        glColor3f(1, 0, 0);
        glVertex3f(-50, -50, 0);
        glVertex3f(50, -50, 0);
        glVertex3f(50, 50, 0);
        glVertex3f(-50, 50, 0);
    glEnd();
    glutSwapBuffers();
}
```

Notes

- Translate **first**, then rotate: the square spins around its own center.
- **glLoadIdentity()** resets the matrix every frame.

glPushMatrix & glPopMatrix

Save and restore

- **glPushMatrix()** saves the current matrix.
- **glPopMatrix()** restores it.
- Use it to transform **one object only**.

```
glPushMatrix();          // save
   glTranslatef(fXPos, fYPos, 0);
   glRotatef(fRot, 0, 0, 1);
   // draw the square here
glPopMatrix();           // restore
```

Use **either** **glLoadIdentity()** or push/pop around the drawing.

Complete Program (1/2)

```
#include <windows.h>
#include <stdlib.h>
#include <gl/gl.h>
#include <gl/glut.h>
float fXPos = 0, fYPos = 0, fRot = 0;
void InitGraphics(int argc, char *argv[]);
void SetTransformations();
void OnDisplay();
void OnKeyPress(unsigned char key, int x, int y);
void OnSpecialKeyPress(int key, int x, int y);
int main(int argc, char* argv[]) {
    InitGraphics(argc, argv);
    return 0;
}
```

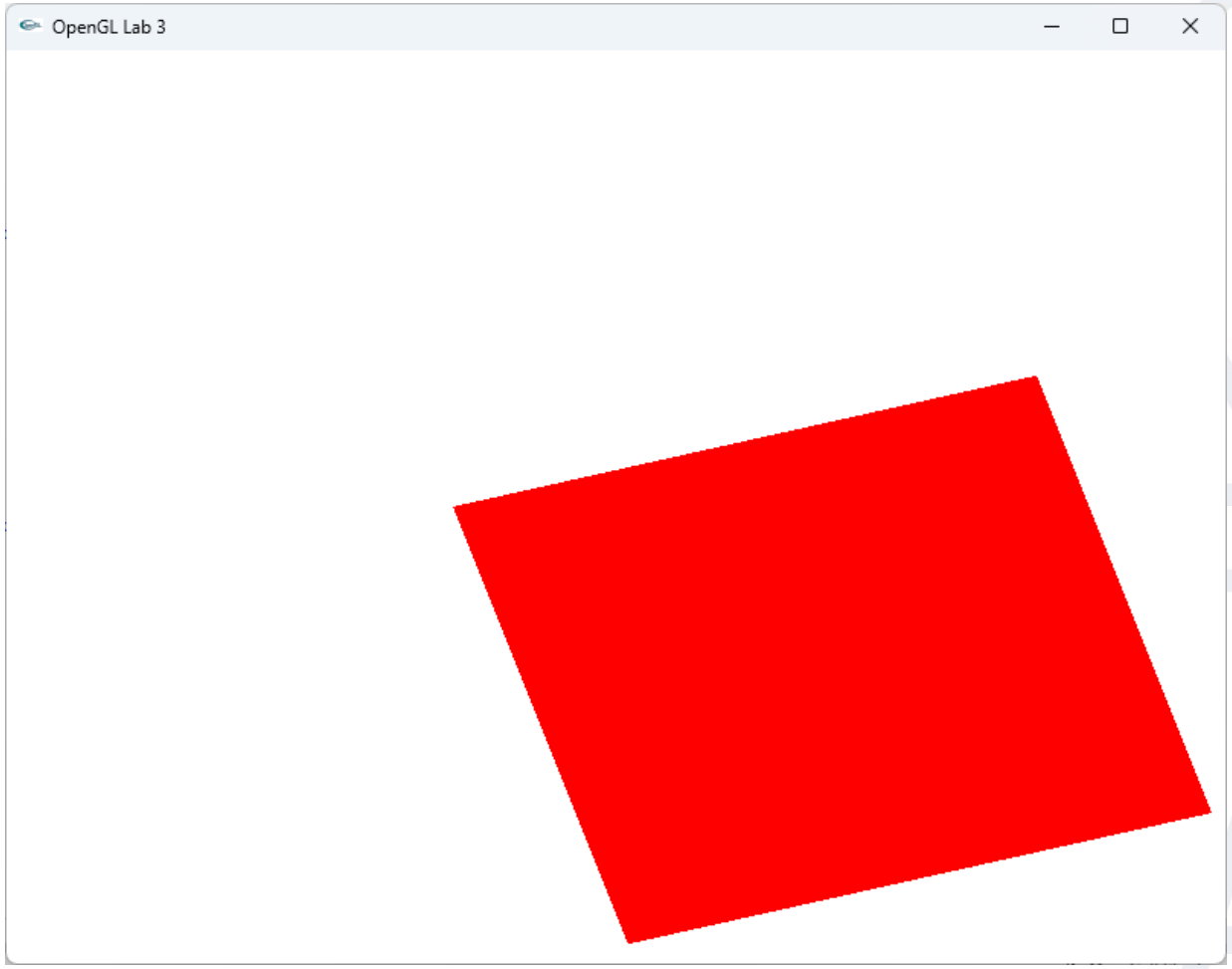
```
void InitGraphics(int argc, char *argv[]) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA);
    glutInitWindowPosition(100, 100);
    glutInitWindowSize(800, 600);
    glutCreateWindow("OpenGL Lab 3");
    glutDisplayFunc(OnDisplay);
    glutIdleFunc(OnDisplay);
    glutKeyboardFunc(OnKeyPress);
    glutSpecialFunc(OnSpecialKeyPress);
    SetTransformations();
    glutMainLoop();
}
void SetTransformations() {
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-100, 100, -100, 100);
    glMatrixMode(GL_MODELVIEW);
}
```

Complete Program (2/2)

```
void OnDisplay() {
    glLoadIdentity();
    glClearColor(1, 1, 1, 1);
    glClear(GL_COLOR_BUFFER_BIT);
    glTranslatef(fXPos, fYPos, 0);
    glRotatef(fRot, 0, 0, 1);
    glBegin(GL_QUADS);
        glColor3f(1, 0, 0);
        glVertex3f(-50, -50, 0);
        glVertex3f(50, -50, 0);
        glVertex3f(50, 50, 0);
        glVertex3f(-50, 50, 0);
    glEnd();
    glutSwapBuffers();
}
```

```
void OnKeyPress(unsigned char key, int x, int y) {
    if (key == 27) exit(0);
    switch (key) {
    case 'a': case 'A': fXPos -= 0.5; break;
    case 'd': case 'D': fXPos += 0.5; break;
    case 'w': case 'W': fYPos += 0.5; break;
    case 's': case 'S': fYPos -= 0.5; break;
    case 'e': case 'E': fRot += 0.1;  break;
    case 'q': case 'Q': fRot -= 0.1;  break;
    }
}
void OnSpecialKeyPress(int key, int x, int y) {
    switch (key) {
    case GLUT_KEY_LEFT:  fXPos -= 0.5; break;
    case GLUT_KEY_RIGHT: fXPos += 0.5; break;
    case GLUT_KEY_UP:    fYPos += 0.5; break;
    case GLUT_KEY_DOWN:  fYPos -= 0.5; break;
    }
}
```

Run: Complete Project



Any Question



THANK YOU

QUESTIONS?





EELU

الجامعة المصرية للتعلم الإلكتروني الأهلية

THE EGYPTIAN E-LEARNING UNIVERSITY